

The Unified Text-to-Text Transfer Transformer: A Technical Treatise on Architecture, Pre-training, and Multilingual Scaling

The emergence of the Text-to-Text Transfer Transformer (T5) marked a pivotal transition in the landscape of natural language processing, moving the field away from task-specific architectural modifications toward a unified representation of all linguistic problems as a sequence-to-sequence task.¹ Developed by researchers at Google AI, the T5 framework is not merely a model but a comprehensive systematization of the transfer learning paradigm.³ By converting every input into a text string and training the model to generate a corresponding target string, the researchers effectively harmonized the training objectives for classification, regression, translation, and summarization.¹ This report provides an exhaustive technical analysis of the T5 and mT5 architectures, their mathematical foundations, and the empirical insights derived from the large-scale ablation studies that defined their development.

The Core Paradigm: Text-to-Text as a Unifying Interface

The primary innovation of T5 is the conceptualization of natural language processing as a "text-to-text" problem.¹ Historically, the application of transfer learning to diverse tasks required structural changes to the model's output layers.² For instance, a BERT-based model for sentiment analysis would utilize a linear classifier on top of the encoder's token, while a version for question answering would require two separate output heads to predict the start and end indices of the answer span within the input.²

The text-to-text formulation eliminates this architectural heterogeneity.¹ In the T5 framework, a sentiment analysis task is handled by training the model to generate the literal word "positive" or "negative" as a text string.⁴ Similarly, regression tasks are solved by training the model to output the string representation of a numerical value.⁴ This approach allows for a singular, consistent training procedure, loss function, and decoding mechanism across all tasks.³

Disruption of Traditional NLP Pipelines

Traditional pipelines often involved a multi-stage process of feature extraction followed by task-specific supervised learning.¹ While models like GPT and BERT improved this by providing powerful pre-trained encoders or decoders, the final stage remained fragmented.² T5's power lies in its ability to internalize task identification through the use of task-specific prefixes.³ By

prepending a string such as "summarize:" or "translate English to German:" to the input, the model is conditioned to trigger the appropriate latent transformation required for the target task.³

This unified approach enables a more rigorous comparison of transfer learning techniques because it controls for architectural variance.³ It allows researchers to evaluate whether a specific pre-training objective or dataset is inherently better across all tasks, rather than being optimized for a specific downstream configuration.¹

Transformer Architecture from First Principles

The T5 model is built upon the classic encoder-decoder Transformer architecture, diverging from the trend of using encoder-only (e.g., BERT) or decoder-only (e.g., GPT) structures.⁸ The empirical evidence gathered during the development of T5 suggested that the encoder-decoder framework, while possessing more parameters for a given layer depth, provides the most flexible and robust performance for text-to-text tasks.³

The Encoder-Decoder Structure

The encoder component of T5 is tasked with producing a contextualized representation of the entire input sequence.⁴ It utilizes a bidirectional attention mechanism, allowing each input token to attend to every other input token.⁸ The decoder, conversely, operates autoregressively, generating tokens one at a time while attending to both the previously generated tokens (via causal self-attention) and the complete encoder output (via cross-attention).⁵

In the T5 implementation, the encoder and decoder are typically symmetrical in terms of the number of layers and hidden dimensionality.⁸ The table below provides a detailed breakdown of the architectural parameters for the five standard T5 model sizes.

Model	Parameters	L (Layers)	dmodel	dff	dkv	h (Heads)
Small	60 million	6	512	2048	64	8
Base	220 million	12	768	3072	64	12
Large	770 million	24	1024	4096	64	16

3B	3 billion	24	1024	16384	128	32
11B	11 billion	24	1024	65536	128	128

8

A critical observation in the scaling of T5 is the non-standard expansion of the feed-forward network (d_{ff}) in the 3B and 11B variants.⁸ Unlike the smaller models where $d_{ff} = 4 \times d_{model}$, the 11B model utilizes a d_{ff} of 65,536, which is 64 times the hidden dimension.⁸ This scaling strategy increases the model's capacity for factual knowledge storage within the MLP layers while maintaining a manageable computational cost for the self-attention mechanism.⁸

Tokenization and Embedding Space

T5 utilizes a WordPiece tokenizer with a standard vocabulary size of 32,000 tokens for its English-centric variants.⁸ The tokenizer is trained on a mixture of languages from the C4 dataset to provide basic multilingual coverage, although its primary focus remains English.⁸ The model employs "tied embeddings," meaning the same weight matrix is shared between the encoder's input, the decoder's input, and the final output projection layer.¹³ To maintain numerical stability and representation quality when using tied embeddings, T5 rescales the embeddings by $\sqrt{d_{model}}$ before feeding them into the first Transformer block.¹³

The Attention Mechanism and Relative Position Bias

The self-attention mechanism in T5 follows the multi-head formulation, where the input is projected into h heads, each with a dimensionality of d_{kv} .⁸ For a given head, the queries (Q), keys (K), and values (V) are computed via linear transformations of the input $X \in \mathbb{R}^{n \times d_{model}}$: $Q = XW_Q$, $K = XW_K$, $V = XW_V$ where $W_Q, W_K, W_V \in \mathbb{R}^{d_{model} \times d_{kv}}$.¹³

The core mathematical formulation of T5's attention is defined by the inclusion of a relative position bias B :

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T + B}{\sqrt{d_k}} \right) V$$

.⁵

Unlike the original Transformer which used absolute positional encodings added to the input embeddings, T5 adopts a "relative scalar bias" approach.⁵ This bias B is a learnable parameter that depends on the relative distance between the attending token i and the attended-to token j .⁵

Relative Position Bucketing Logic

To enable the model to generalize to sequence lengths beyond those seen during training, T5 groups relative distances into a fixed number of buckets.¹⁶ This prevents the model from needing a unique parameter for every possible integer distance.¹⁶ The bucketing logic provides high resolution for nearby tokens while using a logarithmic scale for distant tokens, reflecting the intuition that exact relative positioning is more critical for local context than for long-range dependencies.¹⁷

The mapping of a relative distance $d = j - i$ to a bucket index b follows a specific algorithm:

1. **Linear Range:** For small distances, the bucket index is simply the distance itself.¹⁷
2. **Logarithmic Range:** For distances beyond the linear threshold, the bucket index is computed as:

$$b = \text{max_linear} + \text{floor} \left(\frac{\log(|d|/\text{max_linear})}{\log(\text{max_dist}/\text{max_linear})} \times (\text{num_buckets} - \text{max_linear}) \right)$$

.¹⁷

3. **Maximum Cutoff:** All distances greater than a max_dist (default 128) are assigned to the final bucket.¹⁷

These relative biases are shared across all layers of the model, though each head within a layer maintains its own unique set of biases.⁵ This allows the model to develop different spatial attention patterns—some heads focusing on local structure while others attend to broader, long-range context—without the rigid constraints of absolute position embeddings.⁵

Root Mean Square Layer Normalization (RMSNorm)

T5 introduces a streamlined version of Layer Normalization that omits the mean subtraction and the additive bias term.⁴ This is known as Root Mean Square Layer Normalization

(RMSNorm).¹³ The output y for an input vector x is calculated as:

$$y_i = \frac{x_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}} \times \gamma_i$$

where γ is a learnable scaling parameter and ϵ is a small constant for numerical stability.⁵

Furthermore, T5 employs "pre-normalization".⁵ In this configuration, the input to each sub-layer (attention or feed-forward) is first normalized by RMSNorm, and the output of the sub-layer is added back to the original input via a residual connection.⁵ This contrast with the "post-normalization" used in the original Transformer helps stabilize the gradients during the training of very deep models by maintaining a clear identity path through the network.⁵

The Span Corruption Training Objective

The pre-training phase of T5 is defined by an unsupervised denoising task that was determined to be superior to standard language modeling or individual token masking.¹ This objective is referred to as "span corruption".¹⁰

Mechanism of Action

In span corruption, the model is presented with a corrupted version of a text sequence where contiguous spans of tokens are replaced by a single unique sentinel token (e.g., <extra_id_0>, <extra_id_1>).¹⁰ The target sequence for the decoder consists of the original tokens that were removed, preceded by the same sentinel tokens and followed by an end-of-sequence marker.¹⁰

For example, if the original text is "The quick brown fox jumps over the lazy dog," and we mask the spans "quick brown" and "lazy," the data flow would be:

- **Encoder Input:** "The <extra_id_0> fox jumps over the <extra_id_1> dog."¹²
- **Decoder Target:** "<extra_id_0> quick brown <extra_id_1> lazy <extra_id_2>"¹²

This objective is highly scalable because the decoder only needs to generate the missing spans, rather than the entire sequence.¹² This results in much shorter target sequences, significantly reducing the computational cost and memory requirements during the training of large-scale models.¹²

Comparison with BERT and GPT Objectives

T5's training differs fundamentally from BERT's Masked Language Modeling (MLM) and GPT's Causal Language Modeling (CLM) ⁶:

- **vs. BERT:** While BERT masks individual tokens and requires the model to predict the

entire sequence at the output, T5's span corruption masks variable-length chunks and utilizes an encoder-decoder to generate only the missing pieces.² This forces the model to learn not just word identities but also the expected length and syntactic structure of missing segments.¹²

- **vs. GPT:** GPT is trained to predict the next token in a sequence given all previous tokens.²¹ While powerful for generation, this unidirectional approach is less efficient for understanding tasks that require global context.⁸ T5 combines the bidirectional context of an encoder with the generative capabilities of a decoder.⁸

Loss Function: Cross-Entropy

The model is trained using a standard teacher-forcing maximum-likelihood objective.⁶ For each token in the target sequence, the model outputs a probability distribution over the vocabulary via a softmax layer.²³ The loss is the cross-entropy between the predicted distribution and the ground truth token.³ Because the "text-to-text" framework allows regression and classification to be cast as text generation, this single loss function is sufficient for all pre-training and fine-tuning tasks.³

mT5: Scaling to 101 Languages

The development of mT5 (Multilingual T5) was motivated by the need to extend T5's capabilities to a global scale.⁶ mT5 retains the architectural core of T5 but introduces several critical changes to handle the linguistic diversity of 101 languages.²⁴

Vocabulary Expansion

The most significant architectural change in mT5 is the expansion of the vocabulary from 32,000 to 250,000 WordPieces.⁶ This massive increase was necessary to provide sufficient coverage for the unique characters and sub-words of diverse scripts, including Hanzi, Cyrillic, and various Brahmic scripts.⁶ To ensure that even the rarest characters could be represented, mT5 utilized SentencePiece with a character coverage of 0.99999 and enabled "byte-fallback," which decomposes unknown characters into their UTF-8 byte representations.⁶

Multilingual Data Sampling (mC4)

mT5 was trained on the mC4 dataset, a multilingual variant of C4 containing trillions of tokens across 101 languages.⁶ To prevent high-resource languages like English and Spanish from completely dominating the model's representations, the researchers employed a

temperature-based sampling strategy.²⁵ The sampling probability for a language L is defined as:

$$p(L) \propto |L|^\alpha$$

where $|L|$ is the total number of examples available for that language and α is a smoothing parameter.²⁵ By setting $\alpha = 0.3$, mT5 "boosted" the sampling rate of low-resource languages, ensuring the model saw enough data to learn their underlying structures without excessively repeating the data of high-resource languages.²⁵

The "Accidental Translation" Problem

A unique challenge identified during the zero-shot evaluation of mT5 was the phenomenon of "accidental translation".⁶ This occurs when a model is fine-tuned for a task like summarization using only English data.⁶ When subsequently asked to summarize a Spanish document, the model might produce a summary that is factually correct but written in English.⁶

This behavior is attributed to the model "forgetting" how to generate non-English text during English-only fine-tuning.⁶ To alleviate this, the researchers introduced a simple but effective technique: mixing a small amount of the original unsupervised multilingual pre-training data into the fine-tuning mixture.⁶ This provides a continuous "language identification" signal that encourages the model to remain in the target language's probability space.⁶

Fine-Tuning vs. Pre-training: Parameter Dynamics

Transfer learning in the T5 framework involves two distinct phases: a massive-scale pre-training phase on C4 or mC4, followed by task-specific fine-tuning.¹

What Changes During Fine-Tuning?

During standard fine-tuning, all parameters of the model are typically updated using a constant learning rate (typically 10^{-4} or $3 \cdot 10^{-4}$).¹ This differs from BERT-style pipelines where the final head is the only part that might be trained from scratch; in T5, the "head" is the entire decoder, which is already pre-trained to produce text.²

At the parameter level, fine-tuning shifts the model's internal representations to favor the specific vocabulary and syntactic patterns of the target task.¹ For example, during pre-training, the model learns a broad "denoising" capability.¹⁰ During fine-tuning on a summarization task, it learns to prioritize salient information over local grammatical reconstruction.⁴

Why Transfer Learning Works

Transfer learning is effective because the pre-training phase allows the model to develop a "world model" and a deep understanding of language structure, syntax, and semantics from a massive, diverse corpus.¹ This foundational knowledge provides a starting point that is significantly closer to the optimal parameters for any specific task than a random initialization.² The "text-to-text" framework maximizes this advantage by ensuring that the fine-tuning task uses the exact same interface as the pre-training task, allowing for the direct reuse of the

decoder's generative capabilities.¹

The Inference Process: From Vectors to Strings

Inference in T5 is a multi-step autoregressive process where the model generates one token at a time until an end-of-sequence (</s>) token is produced.²¹

Step-by-Step Generation

1. **Encoder Pass:** The input text (including the task prefix) is tokenized and passed through the encoder.⁸ The encoder produces a sequence of hidden state vectors representing the contextualized input.⁴
2. **Initial Decoder Step:** The decoder receives a start-of-sequence token (typically the padding token).¹² It performs self-attention on this token and cross-attention on the encoder's output to produce the first hidden state.⁸
3. **Token Prediction:** The final hidden state is projected onto the vocabulary space via the tied embedding matrix, and a softmax function produces a probability distribution over all tokens.¹³
4. **Autoregressive Loop:** The selected token is appended to the decoder's input sequence, and the process repeats.²³ In each step t , the decoder has access to tokens 0 through $t - 1$ through its causal attention mask.⁸

Decoding Strategies: Beam Search vs. Greedy

The choice of decoding strategy has a significant impact on the quality and computational cost of inference.²¹

- **Greedy Decoding:** At each step, the model simply picks the token with the highest probability.²¹ While fast and simple, it is "myopic" and can lead to suboptimal sequences if a high-probability initial choice leads to a low-probability sequence overall.²³ A common failure mode is the "repetition loop," where the model gets stuck repeating a specific phrase.²¹
- **Beam Search:** The model maintains k (the beam width) most likely sequences at each step.²¹ It explores multiple paths simultaneously and ranks them by their cumulative log-probabilities.²¹ This often results in much higher-quality text for tasks like translation and summarization where global coherence is essential.²³

Feature	Greedy Decoding	Beam Search

Selection	Top-1 token	Top- k sequences
Computational Cost	Low (1 forward pass/token)	High (k forward passes/token)
Output Quality	Prone to local maxima	Higher coherence
Task Suitability	Simple QA, classification	Summarization, Translation

23

Strengths and Technical Limitations

The T5 and mT5 models represent a high-water mark for Transformer-based transfer learning, yet they are not without significant constraints.⁷

Areas of Excellence

T5 excels in tasks that require complex transformations and multi-step reasoning.⁴ It is particularly strong in "closed-book" question answering, where it can generate correct answers to factual questions based solely on the knowledge internalized during pre-training.² Its 11-billion parameter version achieved state-of-the-art results on benchmarks such as GLUE, SuperGLUE, and SQuAD, often reaching near-human performance levels.²

Failure Modes and Hallucinations

Despite their fluency, T5 models are prone to hallucinations—generating text that is grammatically correct but factually inaccurate or unsupported by the input.³⁰ This is a fundamental risk of the probabilistic next-token prediction paradigm.³² Hallucinations often occur when:

- **Overfitting:** The model becomes too sensitive to patterns in the fine-tuning data, leading it to generate "safe" or biased answers that do not reflect the specific input.³⁰
- **Context Window Limits:** T5's quadratic attention complexity makes it difficult to process very long documents, leading to "contextual hallucinations" where the model ignores distant but relevant information.³³
- **Conflicting Sources:** When a model is provided with long context containing noisy or

contradictory info, it may merge sources incorrectly, creating a plausible but false hybrid.³³

Practical Insight: Navigating Hardware and Scaling Trade-offs

Choosing the right T5 or mT5 variant requires a careful evaluation of available hardware resources and performance requirements.⁷

Hardware Requirements for Fine-Tuning

Full fine-tuning of large language models is exceptionally memory-intensive.³⁵ A standard "rule of thumb" for practitioners is that full fine-tuning requires approximately 16GB of VRAM per 1 billion parameters.³⁵ This is because the GPU must store not only the model weights but also the gradients and the multiple copies of parameters required by optimizers like AdamW.³⁶

Model	Size	VRAM (Full Fine-Tuning)	VRAM (Inference/LoRA)
T5-Base	220M	~4 GB	~1 GB
T5-Large	770M	~12 GB	~2-3 GB
T5-3B	3B	~48 GB	~8-10 GB
T5-11B	11B	~176 GB	~24-30 GB

35

For users with limited hardware, such as a single 24GB consumer GPU (RTX 3090/4090), the 3B or 11B models can only be trained using parameter-efficient techniques like LoRA or QLoRA.³⁵ LoRA (Low-Rank Adaptation) freezes the main weights and only trains a small set of auxiliary matrices, reducing memory requirements for optimizer states by orders of magnitude.³⁵

Choosing mT5 for Limited Hardware

If working with a multilingual dataset on limited hardware, mT5-Small or mT5-Base are often preferred over their English T5 counterparts.²⁴ While they have more total parameters (300M vs 60M for T5-Small), a vast majority of those parameters are in the embedding layer.²⁵ This means the "computational" part of the model (the Transformer blocks) remains small, allowing for reasonably fast training even on mid-tier GPUs.²⁷ However, the large vocabulary does impose a higher fixed memory cost for storing the embedding matrix.²⁵

Scaling vs. Performance

The systematic study of T5 confirmed that scaling—increasing both the model size and the amount of pre-training data—consistently leads to better performance.¹ However, the returns on scaling follow a logarithmic pattern; the jump from T5-Base to T5-Large often yields a more significant relative improvement than the jump from T5-3B to T5-11B.² For many production applications, T5-Base or T5-Large offer the best balance of accuracy and inference latency.⁴

In conclusion, the Text-to-Text Transfer Transformer successfully formalized the intuition that language tasks are fundamentally about transformation.¹ By combining an encoder-decoder architecture with relative position biases and the innovative span corruption objective, T5 provided a robust and scalable blueprint for the next generation of transfer learning.³ Its extension into mT5 further demonstrated the model's versatility, enabling a single unified framework to serve as the foundation for the world's diverse linguistic landscape.²⁴

Works cited

1. Exploring the Limits of Transfer Learning with a Unified Text-to-Text ..., accessed April 29, 2026, <https://jmlr.org/papers/volume21/20-074/20-074.pdf>
2. Exploring Transfer Learning with T5: the Text-To-Text Transfer Transformer - Google Research, accessed April 29, 2026, <https://research.google/blog/exploring-transfer-learning-with-t5-the-text-to-text-transfer-transformer/>
3. T5: Exploring Transfer Learning Limits - Hunter Heidenreich, accessed April 29, 2026, <https://hunterheidenreich.com/notes/natural-language-processing/language-models/t5-text-to-text-transfer-transformer/>
4. T5 - Accubits, accessed April 29, 2026, <https://accubits.com/large-language-models-leaderboard/t5/>
5. T5 Architecture Explained & Encoder-Decoder Model Comparison - AIML.com, accessed April 29, 2026, <https://aiml.com/t5-architecture-explained-encoder-decoder-model-comparison/>
6. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer - ACL Anthology, accessed April 29, 2026, <https://aclanthology.org/2021.naacl-main.41.pdf>
7. Understanding the T5 Model: A Comprehensive Guide | by Gagan Gupta |

- Medium, accessed April 29, 2026,
https://medium.com/@gagangupta_82781/understanding-the-t5-model-a-comprehensive-guide-b4d5c02c234b
8. T5 (language model) - Wikipedia, accessed April 29, 2026,
[https://en.wikipedia.org/wiki/T5_\(language_model\)](https://en.wikipedia.org/wiki/T5_(language_model))
 9. T5 - Hugging Face, accessed April 29, 2026,
https://huggingface.co/docs/transformers/v4.13.0/model_doc/t5
 10. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer | Paper Notes | Tin Rabzelj, accessed April 29, 2026,
<https://rabzelj.com/blog/exploring-the-limits-of-transfer-learning-with-a-unified-text-to-text-transformer-paper-notes>
 11. T5 - Hugging Face, accessed April 29, 2026,
https://huggingface.co/docs/transformers/model_doc/t5
 12. T5 - Harold Benoit, accessed April 29, 2026,
<https://haroldbenoit.com/notes/ml/lms/scaling/t5>
 13. ztjhz/t5-jax: JAX implementation of the T5 model: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer - GitHub, accessed April 29, 2026, <https://github.com/ztjhz/t5-jax>
 14. Formula for Applying T5 Relative Position Bias - 1Cademy, accessed April 29, 2026,
<https://1cademy.com/node/formula-for-applying-t-relative-position-bias/OmRVs qJT9afW58e7dcRY>
 15. The Impact of Positional Encoding on Length Generalization in Transformers - arXiv, accessed April 29, 2026, <https://arxiv.org/pdf/2305.19466>
 16. T5 Bias for Relative Positional Embedding - 1Cademy, accessed April 29, 2026,
<https://1cademy.com/node/t-bias-for-relative-positional-embedding/OTUTR57PSi JwifnPgdog>
 17. T5 relative positional embedding - GitHub Gist, accessed April 29, 2026,
<https://gist.github.com/huchenxucs/c65524185e8e35c4bcfae4059f896c16>
 18. [D] Different Position Encodings, How they work and the ideas behind them - Reddit, accessed April 29, 2026,
https://www.reddit.com/r/learnmachinelearning/comments/1dojth5/d_different_position_encodings_how_they_work_and/
 19. Span Corruption Pre-Training - Emergent Mind, accessed April 29, 2026,
<https://www.emergentmind.com/topics/span-corruption-pre-training>
 20. [Literature Review] SpacTor-T5: Pre-training T5 Models with Span Corruption and Replaced Token Detection - Moonlight | AI Colleague for Research Papers, accessed April 29, 2026,
<https://www.themoonlight.io/en/review/spactor-t5-pre-training-t5-models-with-span-corruption-and-replaced-token-detection>
 21. LLM Inferencing: Greedy Search and Beam Search — Practical Example | by Sulbha Jain, accessed April 29, 2026,
<https://sulbhajain.medium.com/llm-inferencing-greedy-search-and-beam-search-practical-example-b0df89d79296>
 22. The need for sampling temperature and differences between whisper, GPT-3, and

- probabilistic model's temperature | Shivam Mehta, accessed April 29, 2026, <https://shivammehta25.github.io/posts/temperature-in-language-models-open-ai-whisper-probabilistic-machine-learning/>
23. Transformer Sequence Generation | CodeSignal Learn, accessed April 29, 2026, <https://codesignal.com/learn/courses/bringing-transformers-to-life-training-inference/lessons/transformer-sequence-generation>
 24. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer - ACL Anthology, accessed April 29, 2026, <https://aclanthology.org/2021.naacl-main.41/>
 25. mT5: Multilingual T5, accessed April 29, 2026, <https://anwarvic.github.io/cross-lingual-lm/mt5>
 26. XLM-V: Overcoming the Vocabulary Bottleneck in Multilingual Masked Language Models, accessed April 29, 2026, <https://arxiv.org/html/2301.10472>
 27. Papers Explained 113: mT5 - Ritvik Rastogi, accessed April 29, 2026, <https://ritvik19.medium.com/papers-explained-113-mt5-c61e03bc9218>
 28. Gen AI Interview #11: Greedy Decoding vs Beam Search - How LLMs Choose Their Next Word Asked in META, accessed April 29, 2026, <https://www.youtube.com/watch?v=WSMRKjX6CrA>
 29. Beam Search vs Greedy Decoding: LLM Tradeoffs for Production and Interviews - YouTube, accessed April 29, 2026, https://www.youtube.com/watch?v=_JPsgbiYZW0
 30. Large Language Models Hallucination: A Comprehensive Survey - arXiv, accessed April 29, 2026, <https://arxiv.org/html/2510.06265v3>
 31. It's 2026. Why Are LLMs Still Hallucinating? - Duke University Libraries Blogs, accessed April 29, 2026, <https://blogs.library.duke.edu/blog/2026/01/05/its-2026-why-are-llms-still-hallucinating/>
 32. Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior - PMC, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12518350/>
 33. Context Window Limits: Why Your LLM Still Hallucinates - Perivitta Rajendran, accessed April 29, 2026, <https://pr-peri.github.io/llm/2026/02/13/why-hallucination-happens.html>
 34. Towards long context hallucination detection - Amazon Science, accessed April 29, 2026, <https://www.amazon.science/publications/towards-long-context-hallucination-detection>
 35. GPU Options for Finetuning Large Models: Choose the Right Setup | DigitalOcean, accessed April 29, 2026, <https://www.digitalocean.com/resources/articles/gpu-options-finetuning>
 36. How much VRAM do I need for LLM model fine-tuning? - Modal, accessed April 29, 2026, <https://modal.com/blog/how-much-vram-need-fine-tuning>
 37. The Complete Guide to GPU Requirements for LLM Fine-Tuning | Runpod Blog, accessed April 29, 2026, <https://www.runpod.io/blog/llm-fine-tuning-gpu-guide>
 38. Guide to Hardware Requirements for Training and Fine-Tuning Large Language Models, accessed April 29, 2026,

<https://towardsai.net/p/artificial-intelligence/guide-to-hardware-requirements-for-training-and-fine-tuning-large-language-models>

APPROVED - 2026-05-16

APPROVAL CERTIFICATE

E-Flow Document Management System

Document: 5454
Generated: 5/16/2026, 9:51:57 PM
Document ID: 39

APPROVAL CHAIN